

# Project Blueprint: AI-Powered Career Adviser System

**Frontend:** Next.js 14 (React) | **Backend:** FastAPI (Python) | **Database:** PostgreSQL

## 1. Executive Summary

The **Career Adviser System** is a full-stack web application designed to help students navigate their professional future. Unlike traditional static career tests, this system leverages **Artificial Intelligence (NLP)** to analyze resumes, **Data Visualization** to display market trends, and **Interactive Graphs** to generate personalized learning roadmaps.

### High-Level Architecture

- 1. **Frontend (Next.js):** Handles user interaction, state management (Zustand), and visualizations (Recharts/React Flow).
- 2. **Backend (FastAPI):** Manages business logic, authentication, and heavy-lifting Python tasks (Resume Parsing, Data Analysis).
- 3. **Database (PostgreSQL):** Stores relational data including user profiles, career paths, and progress tracking.

## 2. Project Directory Structure

Recommended folder structure to keep the codebase clean and modular.

```
/career-adviser-app
├── /backend                                     # FastAPI Server
│   ├── /app
│   │   ├── /api                               # Route Handlers
│   │   │   ├── auth.py                       # Login/Signup routes
│   │   │   ├── users.py                     # User profile routes
│   │   │   ├── careers.py                   # Career & Roadmap routes
│   │   │   └── admin.py                     # Admin management routes
│   │   ├── /core                             # Security & Config
│   │   │   ├── config.py
│   │   │   └── security.py                 # JWT & Password Hashing
│   │   ├── /db                               # Database
│   │   │   ├── base.py                     # SQLAlchemy Base
│   │   │   └── models.py                   # Database Tables
│   │   ├── /services                         # Business Logic
│   │   │   ├── resume_parser.py            # PDF Extraction logic
│   │   │   └── recommendation.py          # Quiz Algorithm
│   │   └── main.py                           # App Entry Point
│   ├── requirements.txt
│   └── .env
└── /frontend                                 # Next.js Client
```

|  |  |  |                      |                          |
|--|--|--|----------------------|--------------------------|
|  |  |  | /app                 | # App Router             |
|  |  |  | (auth)/login         | # Login Page             |
|  |  |  | dashboard/page.tsx   | # User Dashboard         |
|  |  |  | admin/page.tsx       | # Admin Panel            |
|  |  |  | career/[id]/page.tsx | # Dynamic Career Details |
|  |  |  | /components          | # UI Components          |
|  |  |  | /charts              | # Recharts Components    |
|  |  |  | /roadmap             | # React Flow Graphs      |
|  |  |  | Navbar.tsx           |                          |
|  |  |  | /lib                 | # Utilities              |
|  |  |  | api.ts               | # Axios Instance         |
|  |  |  | /store               | # Zustand Store          |
|  |  |  | package.json         |                          |

### 3. Comprehensive Feature List

#### Phase 1: Core Functionalities (MVP)

*Essential features required for a passing grade.*

##### 1. Authentication System

- **User Registration:** Secure signup with email validation.
- **Login:** JWT (JSON Web Token) based authentication.
- **Password Security:** Passwords hashed using bcrypt before storage.

##### 2. The Interest Assessment Engine

- **Input:** Users take a 15-question quiz ranking their interest in various fields (Math, Art, Logic, People).
- **Algorithm:** A Python weighted scoring system.
  - *Example:* If Creative\_Score > 8 AND Tech\_Score > 5 -> Recommend "UI/UX Designer"
- **Output:** Returns top 3 matched careers with a "% compatibility" score.

##### 3. Career Knowledge Hub

- **Job Details:** Static pages showing Salary, Required Education, and Stress Levels.
- **Resource Aggregator:** Links to external courses (Coursera, Udemy) specific to that career.

##### 4. Smart Search & Filtering

- **Functionality:** Users can manually browse the career database.
- **Filters:** Filter by Salary Range, Remote Work Availability, and Industry (e.g., Tech, Health, Finance).

- **Search:** Real-time keyword search for job titles.

## 5. 🏠 Personal User Dashboard

- **Bookmarking:** Users can "Save" careers they are interested in.
- **Status Tracking:** Simple Kanban board to track job application status (e.g., "To Apply", "Applied", "Interviewing").

## 🟡 Phase 2: Advanced Intelligence (Distinction Level)

*Features that leverage Python's data capabilities.*

## 6. 📄 AI Resume Gap Analyzer

- **Functionality:** Users upload their resume (PDF).
- **The Tech:**
  - *Backend:* Uses `pdfplumber` to read text.
  - *NLP:* Uses `spaCy` to extract "Entities" (Skills like Python, SQL, React).
- **The Logic:** Compares User Skills vs. Career Required Skills.
- **Result:** "You are a 70% match. You are missing: Docker and TypeScript."

## 7. 📖 Interactive Learning Roadmap

- **Functionality:** A visual node-based graph showing the path from Beginner to Pro.
- **The Tech:** React Flow (Frontend library).
- **Interactivity:** Users can click nodes to mark them as "Complete." This progress is saved to the database.

## 8. 📊 Real-Time Market Analytics

- **Functionality:** Interactive charts showing data.
- **The Tech:** Recharts (Frontend) + Pandas (Backend).
- **Visuals:**
  - Bar Chart: "Junior vs Senior Salary".
  - Line Chart: "Job Demand Trends (2020-2025)".

## 9. 🛠️ Admin Management Panel

- **Functionality:** A specialized dashboard for Admin users.
- **Capabilities:** CRUD (Create, Read, Update, Delete) operations for Career entries and Roadmaps.
- **Security:** Protected routes accessible only to users with `role="admin"`.

 **Phase 3: Expert Features (Bonus)**

*Extra features to impress the evaluator.*

10.  **AI Mock Interviewer**

- **Functionality:** A chat interface where an AI acts as an interviewer for the specific role selected.
- **Tech:** OpenAI API or local LLM integration.
- **Feedback:** Provides instant feedback on user answers (e.g., "Your answer was good, but mention specific tools").

11. **Automated Cover Letter Generator**

- **Functionality:** Generates a basic cover letter text file based on the user's selected career and profile data.
- **Output:** A downloadable .txt or .pdf file.

12. **Gamification**

- **Badges:** Users earn badges (e.g., "Novice Developer") for completing roadmap steps.
- **Progress Tracking:** A circular progress bar on the dashboard showing overall completion.

**4. Database Schema (SQLAlchemy Models)**

**Table:** users

| Column          | Type         | Description                               |
|-----------------|--------------|-------------------------------------------|
| id              | Integer (PK) | Unique User ID                            |
| email           | String       | User Email (Unique)                       |
| hashed_password | String       | Security Token                            |
| role            | String       | "user" or "admin"                         |
| full_name       | String       | User's Name                               |
| skills          | JSON         | List of skills e.g., [ "Python", "HTML" ] |
| saved_careers   | JSON         | List of Career IDs bookmarked by user     |

Table: careers

| Column          | Type         | Description                           |
|-----------------|--------------|---------------------------------------|
| id              | Integer (PK) | Unique Career ID                      |
| title           | String       | e.g., "Data Scientist"                |
| description     | Text         | Overview of the job                   |
| salary_min      | Integer      | Starting Salary                       |
| salary_max      | Integer      | Senior Salary                         |
| required_skills | JSON         | e.g., ["Python", "Statistics", "SQL"] |
| stress_level    | Integer      | 1-10 Scale                            |

Table: roadmaps

| Column    | Type         | Description                               |
|-----------|--------------|-------------------------------------------|
| id        | Integer (PK) | Unique Roadmap ID                         |
| career_id | Integer (FK) | Links to careers table                    |
| nodes     | JSON         | Data structure for React Flow nodes       |
| edges     | JSON         | Data structure for React Flow connections |

Table: user\_progress

| Column          | Type         | Description                     |
|-----------------|--------------|---------------------------------|
| id              | Integer (PK) | Unique ID                       |
| user_id         | Integer (FK) | Links to users                  |
| career_id       | Integer (FK) | Links to careers                |
| completed_nodes | JSON         | List of node IDs marked as done |

## 5. API Endpoints Specification

### Authentication

- POST /auth/register - Register a new user.
- POST /auth/login - Authenticate and receive Access Token.

### User Profile

- GET /users/me - Get current user details.
- PUT /users/update-skills - Manually add skills to profile.
- POST /users/bookmark/{career\_id} - Save a career to dashboard.

### Intelligence & Logic

- POST /api/quiz-submit - Send quiz answers; Receive Career Recommendations.
- POST /api/resume-analyze - Upload PDF; Receive "Missing Skills" list.
- POST /api/mock-interview/chat - Send user message; Receive AI Interviewer response.

### Data & Content

- GET /careers - List all available careers (supports query params for filtering).
- GET /careers/{id} - Get specific details for a job.
- GET /careers/{id}/roadmap - Get JSON data for the React Flow graph.
- GET /careers/{id}/market-data - Get salary stats for Recharts.

## 6. Implementation Timeline (4 Weeks)

### Week 1: Foundations & Database

- Set up Next.js and FastAPI environments.
- Configure PostgreSQL and Docker.
- Create Database Models (User, Career, Admin).
- Build Login/Signup API and Frontend Forms.

### Week 2: Core Logic (The Quiz & Search)

- Seed the database with 20 sample careers.
- Write the Python algorithm for the "Interest Quiz".
- Implement Search and Filter functionality.
- Create the Quiz UI and Dashboard in Next.js.

### **Week 3: Advanced Intelligence (Resume & Charts)**

- Implement PDF Parsing with `pdfplumber`.
- Create the Logic to compare Resume Skills vs Career Skills.
- Integrate Recharts for Salary Visualization.
- Build Admin Panel for managing data.

### **Week 4: Roadmap & Polish**

- Implement React Flow for the interactive roadmap.
- Implement Mock Interview Chatbot (Basic Version).
- Style the application using Tailwind CSS.
- Final Testing and Bug Fixes